

First Order Logic

Propositional Logic

- ✓ Declarative - pieces of syntax correspond to facts
- ✓ Allows partial/disjunctive/negated information
- ✓ Compositional ($B_{11} \wedge P_{12}$ is derived from B_{11} and P_{12})
- ✓ Context independent
- ✓ Unambiguous
- × Very limited expressive power
 - Cannot say “pits cause breeze in adjacent squares”, have to list this for each square.

Propositional vs. First Order Logic

❑ PL: Deals with facts and propositions that can be true or false

○ P_{21} - “there is a pit in [2, 1]”

❑ FOPL: closer to natural language, assumes the world contains objects and relations

○ **Objects** - $CK, AICourse, R2D2$

○ **Relations** - $Person(CK),$

$Robot(R2D2),$

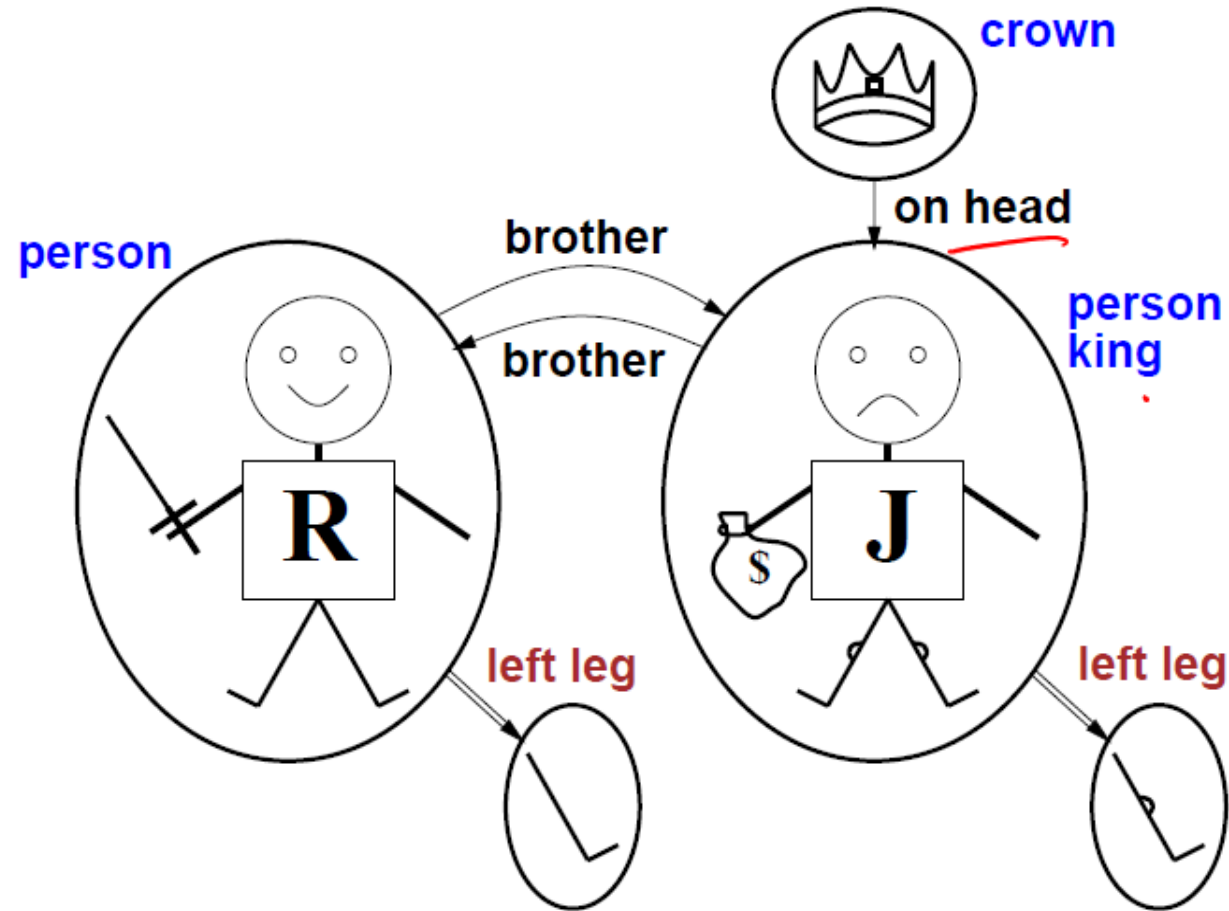
$Teach(CK, AICourse),$

$CSStudent(R2D2),$

$Student(AICourse, R2D2),$

$SleepsInTheClass(R2D2)$

Working Example



FOL - Syntax Definitions

❑ **Constant Symbols:** Name of a specific object

John, Richard, Crown, ...

❑ **Predicate Symbols:** Properties of relationships between objects

- Takes the value true or false
- Argument includes one or more objects

King, Sibling, Person, ...

❑ **Function Symbols:** relations where there is only “value” for a given input

- Mapping from objects to object

Left_Leg_of, Brother_of, ...

FOL - Syntax Definitions (2)

❑ **Terms:** logical expression that refers to an object $f(t_1, t_2, \dots, t_n)$, t_i is a specific object.

Brother_of(John), Left_Leg_of(John)

❑ **Atomic sentence:** is formed from a predicate symbol optionally followed by a parenthesized list of terms

Brother(Richard, John)

- An atomic sentence is true in a given model if the relation referred to by the predicate symbol holds among the objects referred to by the arguments

❑ **Complex sentence:** atomic sentences connected through logical operators

FOL - Syntax Definitions (3)

❑ **Variables:** Refer to an object without naming it

$x, y, z1, \dots$

○ A term with no variables is called a ground term

❑ **Connectives:** Logical connectives to form complex sentences

$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow, =$

❑ **Quantifiers:** express properties of entire collection of objects, instead of exhaustive enumeration

$\forall, \exists$

FOL - Quantifiers

□ $\forall$ - For all - Universal Quantifier

- Statements about every object in the world without naming it
- $\forall x P$ P is true for every object x
- All kings are persons $\forall x King(x) \Rightarrow Person(x)$
- All students in the AI class are smart $\forall x Student(AICourse, x) \Rightarrow Smart(x)$

□ $\exists$ - There exists - Existential Quantifier

- Statement about some object in the world without naming it
- $\exists x P$ P is true for some object x
- John has a crown on his head $\exists x Crown(x) \wedge OnHead(x, John)$
- There is a smart student in the AI class $\exists x Student(AICourse, x) \wedge Smart(x)$

Quantifier-Connective interaction

□ $\forall x \text{ Student}(\text{AICourse}, x) \Rightarrow \text{Smart}(x)$

‘All AI students are smart’

□ $\forall x \text{ Student}(\text{AICourse}, x) \wedge \text{Smart}(x)$

‘Everything is a smart AI student’

□ $\exists x \text{ Student}(\text{AICourse}, x) \Rightarrow \text{Smart}(x)$

‘There exists an object that is either smart, or not an AI student at all’

□ $\exists x \text{ Student}(\text{AICourse}, x) \wedge \text{Smart}(x)$

‘There exists a smart AI student’

Nested Quantifiers

$\forall x \forall y$ is same as $\forall y \forall x$

$\exists x \exists y$ is same as $\exists y \exists x$

$\forall x \exists y$ is not same as $\exists y \forall x$

Every monkey has a tail

$\forall m \exists t \text{ has}(m, t)$

Every monkey shares a tail

$\exists t \forall m \text{ has}(m, t)$

Everybody loves somebody

$\forall x \exists y \text{ loves}(x, y)$

Somebody is loved by everybody

$\exists y \forall x \text{ loves}(x, y)$

$\forall x \left(\text{Crown}(x) \vee (\exists x \text{ Brother}(\text{Richard}, x)) \right)$

The rule is that the variable belongs to the innermost quantifier that mentions it, it will not then be subjected to any other quantification

Connections between $\forall$ and $\exists$

De Morgan's Laws

$\forall x \neg P(x)$	$\equiv$	$\neg \exists x P(x)$
$\neg \forall x P(x)$	$\equiv$	$\exists x \neg P(x)$
$\neg \exists x \neg P(x)$	$\equiv$	$\forall x P(x)$
$\exists x P(x)$	$\equiv$	$\neg \forall x \neg P(x)$

FOL - Semantics

❑ Semantics - What the arrangement of the symbols means in the world.

❑ Propositional Logic

- Basic elements are propositional variables e.g., P_{11}
- Possible worlds - mappings from variables to T/F

❑ First order logic

- Basic elements are terms e.g., Richard, Father_of(Richard)
- **Interpretations:** mappings from terms to real-world elements

Example

Interpretations map syntactic tokens to model elements

Constants:

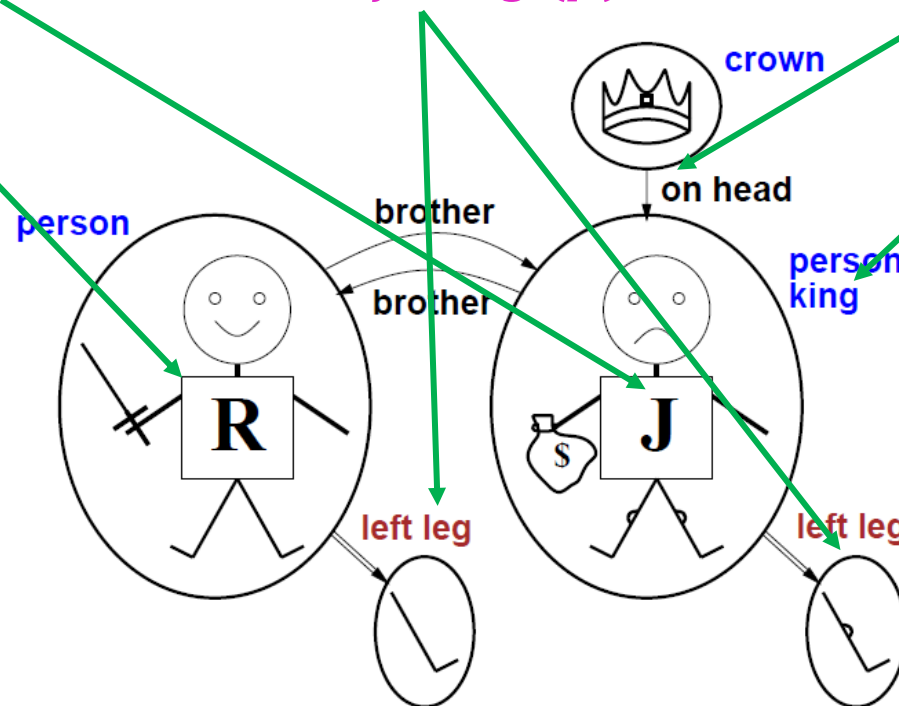
Richard, John

Functions:

LeftLeg(p)

Relations:

On(x, y), King(p)



Example

Interpretations map syntactic tokens to model elements

Constants:

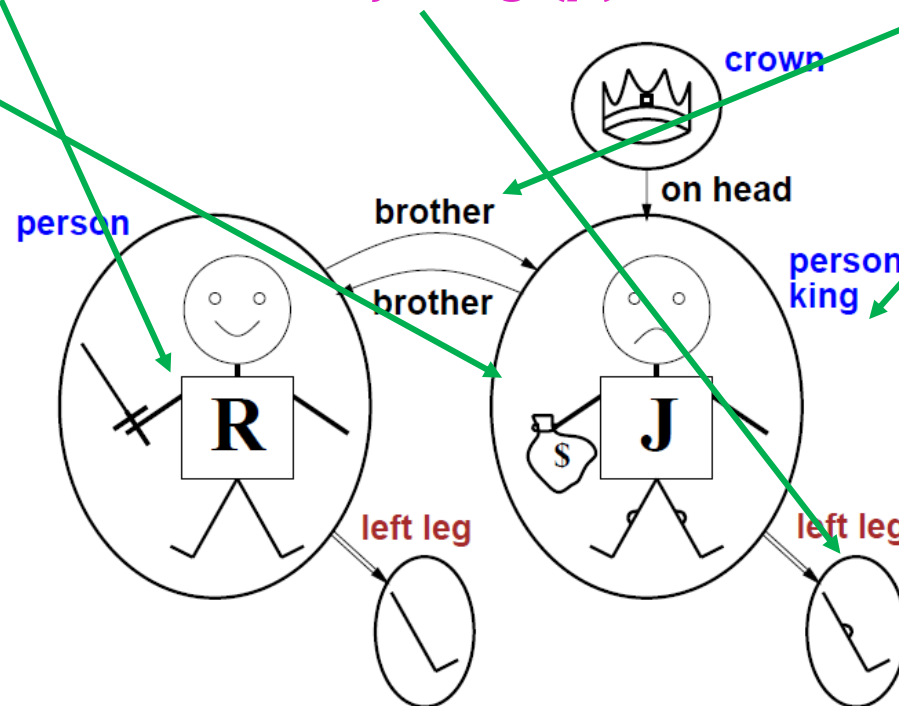
Richard, John

Functions:

LeftLeg(p)

Relations:

On(x, y), King(p)



How many interpretations?

❑ Two constants and 5 objects in the world

- Richard, John (objects: R, J, crown, RL, JL)
- $5^2 = 25$ object mappings

❑ One unary relation

- $King(x)$: x - infinite number of values; infinite number of mappings
- Suppose $x \in \{R, J, Crown, RL, JL\}$, $2^5 = 32$ truth values

❑ Two binary relations

- $Brother(x, y)$; $On(x, y)$
- If we restrict x and y to 5 objects each, yields - 2^{25} mappings for each binary relation.

Satisfiability, Validity and Entailment

- ❑ S is valid if it is true in all interpretations
- ❑ S is satisfiable if it is true in some interpretation
- ❑ S is unsatisfiable if it is false in all interpretations
- ❑ $S_1 \models S_2$ for all interpretations where S_1 is true, S_2 is also true.

PL vs FOL

	PL	FOL
Ontology	Facts (P, Q,...)	objects, relations, functions
Syntax	Atomic sentences connectives	variables, quantifications, terms, ...
Semantics	truth tables	interpretations
inference algorithm	WalkSAT, DPLL	unification, forward/backward chaining, theorem proving
complexity	NP-Complete	semi-decidable, may run forever when no entailment

Examples

- ❑ All men are mortal
- ❑ Socrates is a man
- ❑ Socrates is mortal
- ❑ All purple mushrooms are poisonous
- ❑ A mushroom is poisonous only if it is purple
- ❑ No purple mushroom is poisonous
- ❑ There is exactly one mushroom
- ❑ No human enjoys Golf
- ❑ Some professor who is not a historian writes programs
- ❑ Every boy owns a dog

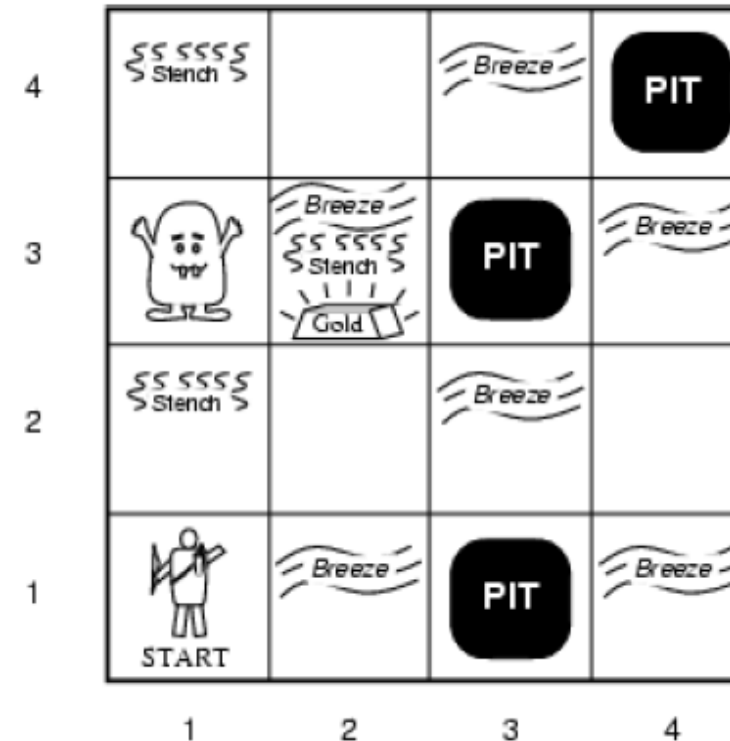
FOL for Wumpus World

□ Objects

- Squares, Wumpus, agent
- Gold, pits, stench, breeze

□ Relations

- Square topology(adjacency)
- Pits/breezes
- Wumpus/stench



Wumpus World - Squares

❑ Each square as an object

- $Square_{11}, Square_{12}, Square_{13}, \dots, Square_{44}$

❑ Adjacency relation

- $Adjacent(Square_{11}, Square_{12}), \dots$
- $Adjacent(Square_{34}, Square_{44})$

❑ Better: **Squares as lists**

- $[1, 1], [1, 2], \dots, [4, 4]$

❑ **Square topology relations**

- $\forall x, y, a, b: Adjacent([x, y], [a, b]) \Leftrightarrow$
 $[a, b] \in \{[x + 1, y], [x - 1, y], [x, y + 1], [x, y - 1]\}$

Wumpus World - Pits

❑ Each pit has an object

- $Pit_{11}, Pit_{21}, \dots, Pit_{44}$
- Problem? Not all squares have pits.

❑ List only the pits we have

- $Pit_{31}, Pit_{33}, Pit_{44}$
- No reason to distinguish between the pits

❑ Pit as a unary predicate

- $Pit(x), Pit([3, 1])$ will be true.

Wumpus World

❑ Breeze: Represent breezes like pits - unary predicate

○ $Breezy(x)$, returns true for squares that are breezy

❑ 'squares adjacent to pits are breezy'

○ $\forall a, b, c, d: Pit([a, b]) \wedge Adjacent([a, b], [c, d]) \Rightarrow Breezy([c, d])$

❑ Agent's locations: changes over time

○ $At(Agent, s, t)$ - agent is in square s at time t .

❑ If the agent is at a square and perceives a breeze, then the square is breezy

○ $\forall s, t (At(Agent, s, t) \wedge Breeze(t)) \Rightarrow Breezy(s)$

Wumpus World

□ 'Breezy implies possible pits in adjacent squares'

○ $\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(s, r) \wedge \text{Pit}(r)$

□ Wumpus - Wumpus as an object

○ *Wumpus*

□ The home square of the Wumpus as a unary predicate

○ *WumpusIn*(*x*)

□ Better: **Wumpus's home as a function**

○ *Home*(*Wumpus*) references Wumpus's home square.

Reasoning and Inference in FOL

□ Instantiation

□ Techniques

- Propositionalization
- Unification
- Forward Chaining
- Backward Chaining
- Resolution

Universal Instantiation

□ Universally quantified sentence

- All students in the AI class are smart $\forall x \text{ Student}(\text{AICourse}, x) \Rightarrow \text{Smart}(x)$

□ Intuitively, x can be anything:

- $\text{Student}(\text{AICourse}, \text{CK}) \Rightarrow \text{Smart}(\text{CK})$
- $\text{Student}(\text{AICourse}, \text{Chair}) \Rightarrow \text{Smart}(\text{Chair})$
- $\text{Student}(\text{AICourse}, \text{LeftLeg}(\text{John})) \Rightarrow \text{Smart}(\text{LeftLeg}(\text{John}))$

□ Formally:

$$\frac{\forall x \quad S}{\text{Subst}(\{x/p\}, S)}$$

- x is replaced with p (ground term) in S , and the quantifier is removed.

Existential Instantiation

□ Existentially Quantified sentence

- There is smart student in the AI class $\exists x \text{ Student}(\text{AICourse}, x) \wedge \text{Smart}(x)$

□ Intuitively, x must name something. But what?

- Can we conclude: $\text{Student}(\text{AICourse}, CK) \wedge \text{Smart}(CK)$???
- No! The sentence might not be true for CK !

□ Instead, Use a **Skolem constant** and draw the conclusion, Formally

$$\frac{\exists x \ S}{\text{Subst}(\{x/K\}, S)}$$

- K is called a Skolem constant and is a completely new symbol you created.

Instantiation

- ❑ Universal instantiation can be applied several times to add new sentences
 - The new KB is logically equivalent to the old
- ❑ Existential instantiation can be applied once to replace the existential sentence
 - The new KB is not equivalent to the old
 - But is satisfiable iff the old KB is satisfiable.

Inference I: Propositionalization

❑ Suppose *KB* contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

King(John)

Greedy(Richard)

Brother(Richard, John)

❑ Instantiating will result in

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

King(John)

Greedy(Richard)

Brother(Richard, John)

❑ The new KB is now propositionalized. What are the proposition symbols?

Inference I: Propositionalization

❑ Suppose *KB* contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

King(John)

Greedy(Richard)

Brother(Richard, John)

❑ Instantiating will result in

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

King(John)

Greedy(Richard)

Brother(Richard, John)

❑ The new KB is now propositionalized. What are the proposition symbols?

King(John), Greedy(John), King(Richard), ...

Propositionalization (2)

- ❑ Every FOL KB and query can be propositionalized in such a way that entailment is preserved.
 - Propositionalize the KB and query
 - Apply resolution!
- ❑ Problems can occur with Function Symbols.
 - e.g., $Father(Father(John))...$
- ❑ Herbrand Theorem: If a sentence α is entailed by an FOL KB , it is entailed by a **finite** subset of the propositional KB .
- ❑ Idea: For $n = 0$ to ∞ do
 - Create a propositional KB by instantiating with depth n - terms.
 - See if α is entailed by this KB .

Works fine if α is entailed, but loops otherwise.
Similar to halting problem in Turing machines - Semidecidable.

Problems with Propositionalization

❑ Can generate lot of irrelevant sentences

❑ e.g., suppose we have

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\forall y \text{ Greedy}(y)$

$\text{Brother}(\text{Richard}, \text{John})$

❑ It seems obvious that $\text{Evil}(\text{John})$, but propositionalization produces lots of facts such as $\text{Greedy}(\text{Richard})$ that are irrelevant

❑ With p k -ary predicates and n constants there are $p \cdot n^k$ instantiations

❑ With function symbols, it gets much worse!!

Propositionalization (3)

- What if we want to use modus ponens from propositional logic?

$$\frac{\alpha \wedge \beta, \alpha \wedge \beta \Rightarrow \gamma}{\gamma}$$

- In FOL?

$$\frac{\forall x \text{ Student}(\text{AICourse}, x) \Rightarrow \text{Smart}(x), \text{Student}(\text{AICourse}, \text{Ram})}{????}$$

- Must 'unify' x with Ram :

- Need to substitute $\{x/\text{Ram}\}$ in $\text{Student}(\text{AICourse}, x) \Rightarrow \text{Smart}(x)$ to infer $\text{Smart}(\text{Ram})$

Unification

❑ Going back, suppose we have

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

King(John)

Greedy(John)

Brother(Richard, John)

❑ We can get the inference immediately if we can find a substitution θ such that

- *King(x)* and *Greedy(x)* match with *King(John)* and *Greedy(John)*
- $\theta = \{x/\text{John}\}$ works

Unification

❑ Going back, suppose we have

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\forall y \text{ Greedy}(y)$

$\text{Brother}(\text{Richard}, \text{John})$

❑ We can still find a substitution θ such that

○ $\text{King}(x)$ and $\text{Greedy}(x)$ match $\text{King}(\text{John})$ and $\text{Greedy}(y)$

○ $\theta = \{x/\text{John}, y/\text{John}\}$ works

❑ $\text{UNIFY}(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

Unification - Most General Unifier

- ❑ Match up expressions by finding variable values that make the expressions identical

Unify $Student(AICourse, x)$ and $Student(AICourse, Ram)$ using $\{x/Ram\}$

- ❑ *Unify*(a, b) returns the most general unifier.
 - That places fewest restrictions on values of variables

Unify($Knows(Ram, x), Knows(y, z)$)

returns $\{y/Ram, x/z\}$ or $\{y/Ram, x/Ram, z/Ram\}$

MGU

- ❑ Unification vs Substitution?

Unification Examples

□ *Unify(Knows(John, x), Knows(John, Jane))*

- *x/Jane*

Unification Examples (2)

❑ *Unify(Knows(John, x), Knows(John, Jane))*

❑ *Unify(Brother(x, John), Brother(Richard, y))*

- *x/Richard, y/John*

Unification Examples (3)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ $Unify(Brother(x, Richard), Brother(John, x))$

Unification Examples (4)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ ~~$Unify(Brother(x, Richard), Brother(John, x))$~~
 - Two sentences happen to use the same variable
 - standardizing apart - renaming the variables in one of the sentences to avoid name clashes. $Unify(Brother(x, Richard), Brother(John, y))$

$x/John, y/Richard$

Unification Examples (5)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ ~~$Unify(Brother(x, Richard), Brother(John, x))$~~
- ❑ $Unify(f(g(x, dog), y), f(g(cat, y), dog))$

$x/cat, y/dog$

Unification Examples (6)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ ~~$Unify(Brother(x, Richard), Brother(John, x))$~~
- ❑ $Unify(f(g(x, dog), y), f(g(cat, y), dog))$
- ❑ $Unify(f(g(x)), f(x))$

Unification Examples (7)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ ~~$Unify(Brother(x, Richard), Brother(John, x))$~~
- ❑ $Unify(f(g(x, dog), y), f(g(cat, y), dog))$
- ❑ ~~$Unify(f(g(x)), f(x))$~~
 - A variable may not contain itself in a substitution
 - occur-check

Unification Examples (8)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ ~~$Unify(Brother(x, Richard), Brother(John, x))$~~
- ❑ $Unify(f(g(x, dog), y), f(g(cat, y), dog))$
- ❑ ~~$Unify(f(g(x)), f(x))$~~
- ❑ $Unify(f(g(cat, y), y), f(x, dog))$
 $x/g(cat, y), y/dog$

Unification Examples (9)

- ❑ $Unify(Knows(John, x), Knows(John, Jane))$
- ❑ $Unify(Brother(x, John), Brother(Richard, y))$
- ❑ ~~$Unify(Brother(x, Richard), Brother(John, x))$~~
- ❑ $Unify(f(g(x, dog), y), f(g(cat, y), dog))$
- ❑ ~~$Unify(f(g(x)), f(x))$~~
- ❑ $Unify(f(g(cat, y), y), f(x, dog))$
- ❑ $Unify(f(g(y)), f(x))$
 $x/g(y)$

Generalized Modus Ponens (GMP)

$\text{Student}(\text{Alcourse}, \text{Ram}) \quad \forall x \text{ Student}(\text{AlCourse}, x) \Rightarrow \text{Smart}(x)$
????

□ Unify x with Ram - $\{x/\text{Ram}\}$ in $\text{Student}(\text{AlCourse}, x) \Rightarrow \text{Smart}(x)$

□ Apply MP to infer $\text{Smart}(\text{Ram})$

□ Generalized Modus Ponens (GMP)

○ 'Lifted' version of MP (lifts MP from ground PI to FOL)

$$\frac{p'_1, p'_2, \dots, p'_n, \quad (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q}$$

□ where $p'_i\theta = p_i\theta$ for all i

GMP used with KB of definite clauses, all variables assumed universally quantified.

Prove GMP is sound!

Inference II: Forward Chaining

```
function FOL-FC-Ask( $KB, \alpha$ ) returns a substitution or false
  repeat until new is empty
     $new \leftarrow \{\}$ 
    for each sentence  $r$  in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$ 
      for each  $\theta$  such that  $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  is not a renaming of a sentence already in  $KB$  or new then do
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false
```

The algorithm:

- Start with the KB
- Add any fact you can generate with GMP (unify + GMP)
- Repeat until goal is reached or generation halts.

Forward Chaining Example

- ❑ It is a crime for an Indian to sell weapons to hostile nations. The country Nono, an enemy of India, has some missiles. All of its missiles were sold to it by Traitor, who is an Indian.
- ❑ Is Traitor a criminal?

Forward Chaining Example

- ❑ It is a crime for an Indian to sell weapons to hostile nations. The country Nono, an enemy of India, has some missiles. All of its missiles were sold to it by Traitor, who is an Indian.
- ❑ Is Traitor a criminal? *Criminal(Traitor)?*
- ❑ KB of definite clauses (exactly one positive literal)
 - $Indian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$
 - $Enemy(Nono, India)$
 - $Owns(Nono, M_1)$
 - $Missile(M_1)$
 - $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Traitor, x, Nono)$
 - $Indian(Traitor)$
 - $Missile(x) \Rightarrow Weapon(x)$
 - $Enemy(x, India) \Rightarrow Hostile(x)$

M_1 is a Skolem constant

Forward Chaining Example

$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Traitor, x, Nono)$

$Missile(x) \Rightarrow Weapon(x)$

$Enemy(x, India) \Rightarrow Hostile(x)$

$Indian(Traitor)$

$Missile(M_1)$

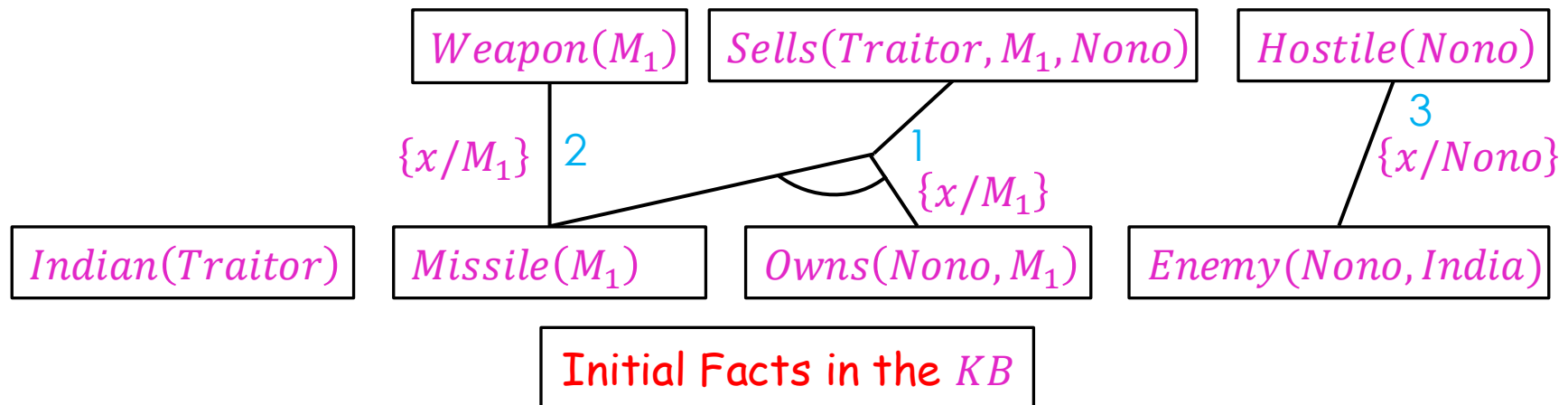
$Owns(Nono, M_1)$

$Enemy(Nono, India)$

Initial Facts in the KB

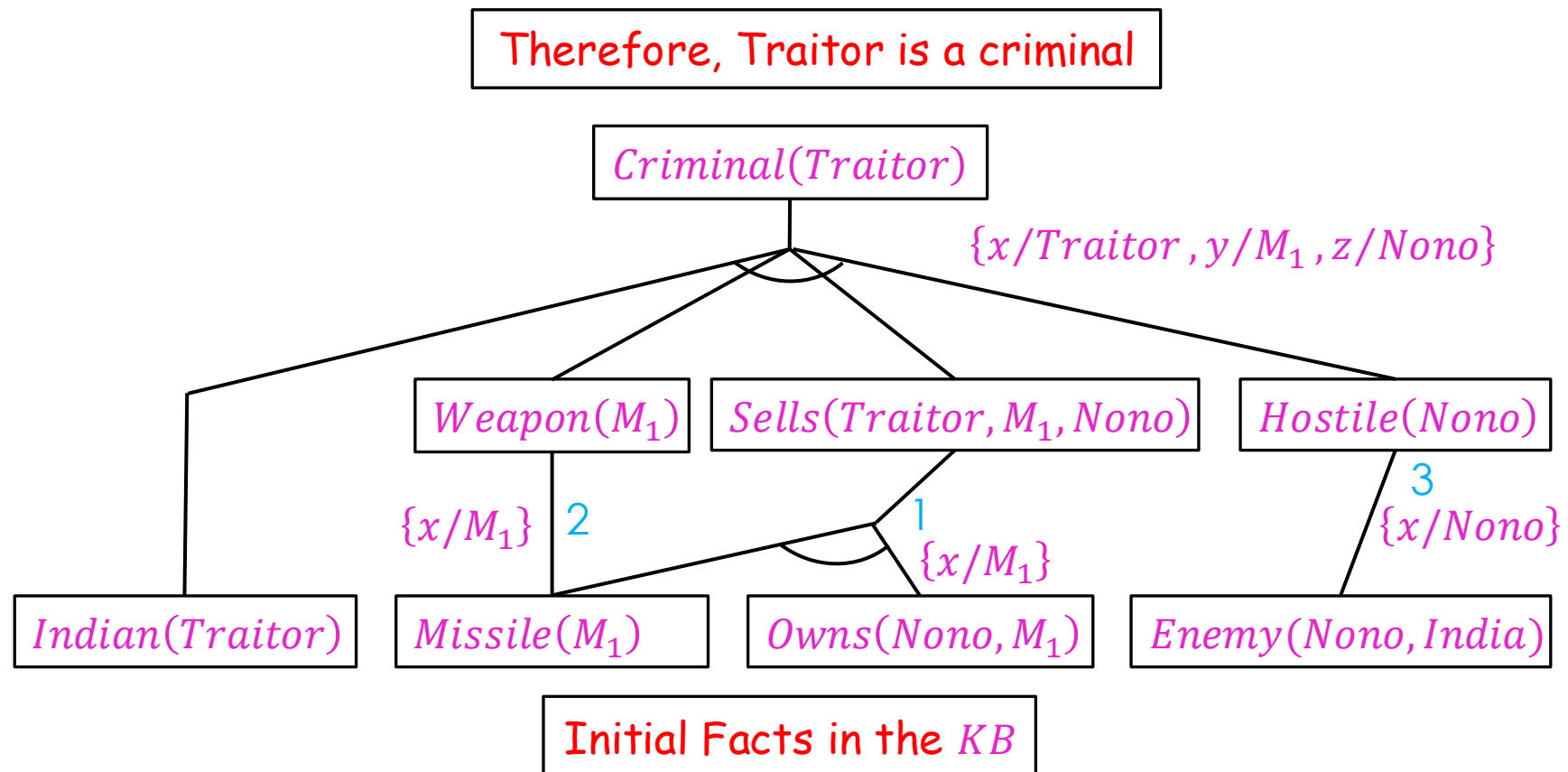
Forward Chaining Example

- 1 $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Traitor, x, Nono)$
- 2 $Missile(x) \Rightarrow Weapon(x)$
- 3 $Enemy(x, India) \Rightarrow Hostile(x)$



Forward Chaining Example

$Indian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$



Inference II: Forward Chaining

❑ Sound

- Yes, Because GMP is sound

Check pg. 331 in textbook

❑ Complete

- Yes, if *KB* contains only definite clauses

❑ Efficiency

- Unification via exhaustive pattern matching
- Rule rechecking for premise satisfaction at every iteration
- Irrelevant fact generation

Check section 9.3.3 for possible strategies

Inference III: Backward Chaining

```
function FOL-BC-ASK(KB, goals,  $\theta$ ) returns a set of substitutions
  inputs: KB, a knowledge base
           goals, a list of conjuncts forming a query ( $\theta$  already applied)
            $\theta$ , the current substitution, initially the empty substitution  $\{ \}$ 
  local variables: answers, a set of substitutions, initially empty
  if goals is empty then return  $\{ \theta \}$ 
   $q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(\textit{goals}))$ 
  for each sentence r in KB
    where  $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$ 
    and  $\theta' \leftarrow \text{UNIFY}(q, q')$  succeeds
     $\textit{new\_goals} \leftarrow [p_1, \dots, p_n | \text{REST}(\textit{goals})]$ 
     $\textit{answers} \leftarrow \text{FOL-BC-ASK}(\textit{KB}, \textit{new\_goals}, \text{COMPOSE}(\theta', \theta)) \cup \textit{answers}$ 
  return answers
```

- Start with KB and goal
- Find all rules whose results unify with goal:
 - Add the premises of these rules to the goal list
 - Remove the corresponding result from the goal list
- Stop when
 - goal list is empty(success) or progress halts (failure)

Backward Chaining Example

Goal

Criminal(Traitor)

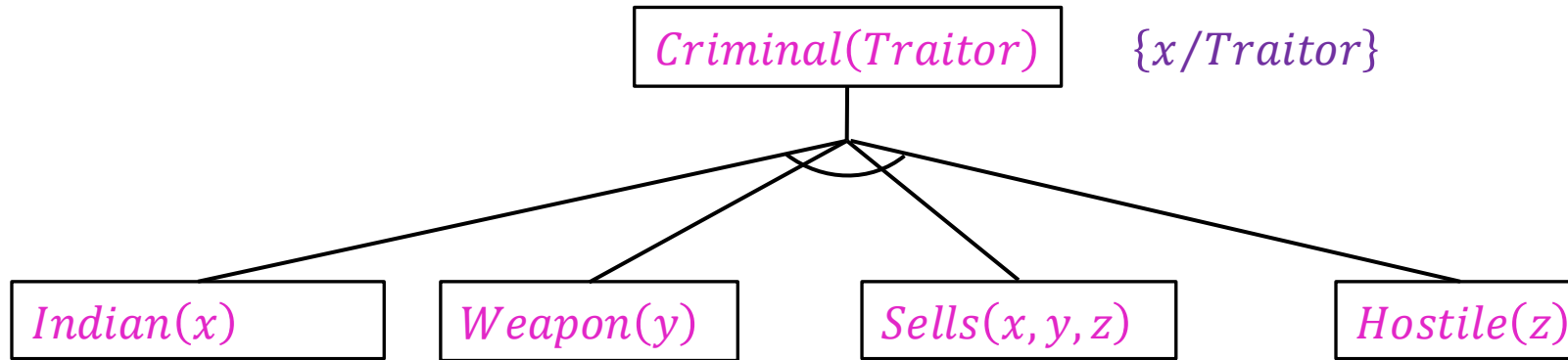
Backward Chaining Example

□ $Indian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

$Criminal(Traitor)$ $\{x/Traitor\}$

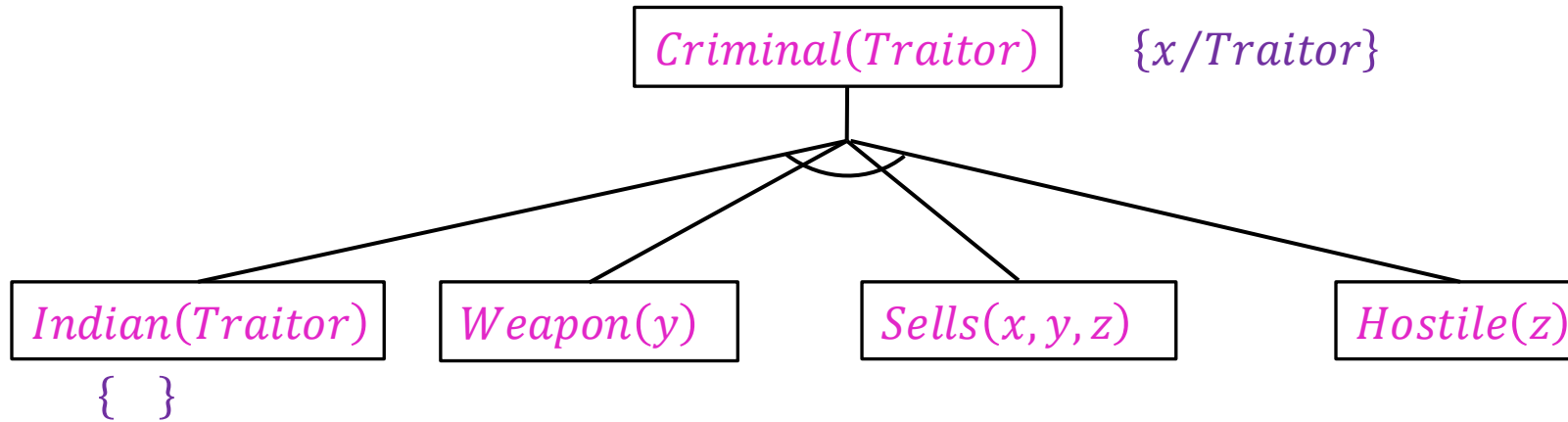
Backward Chaining Example

□ Depth-First Traversal



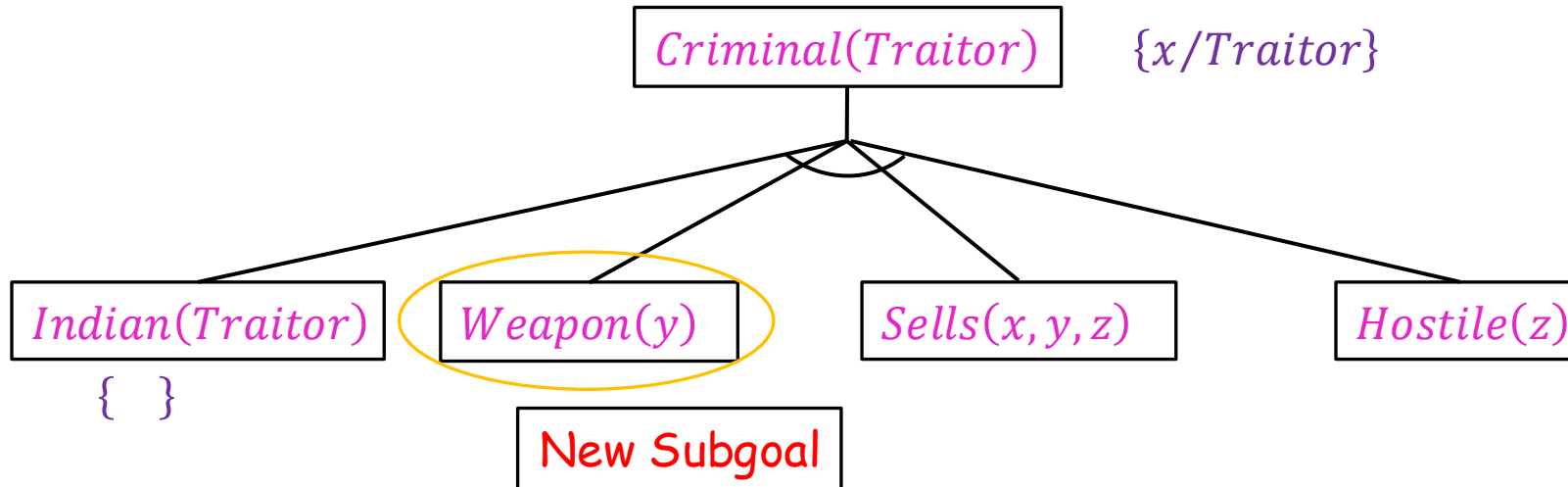
Backward Chaining Example

□ Depth-First Traversal



Backward Chaining Example

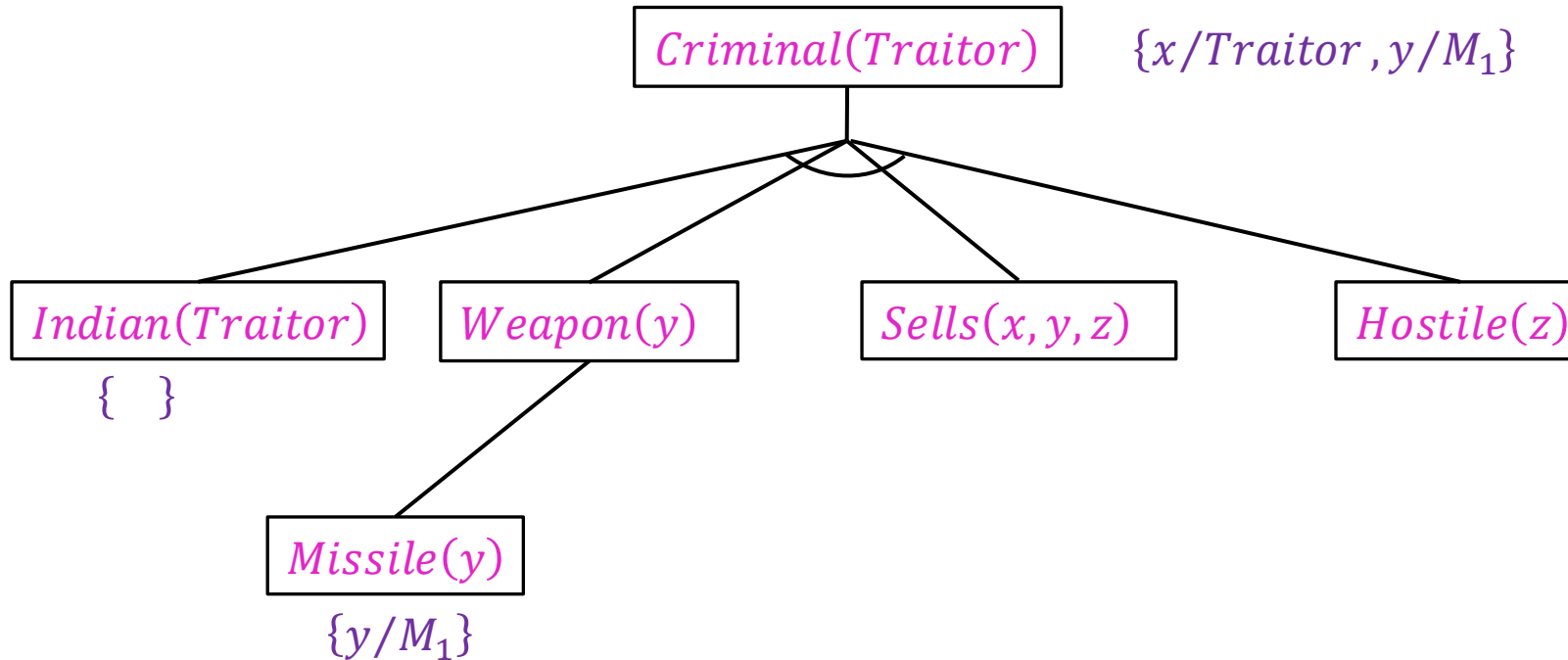
□ Depth-First Traversal



Backward Chaining Example

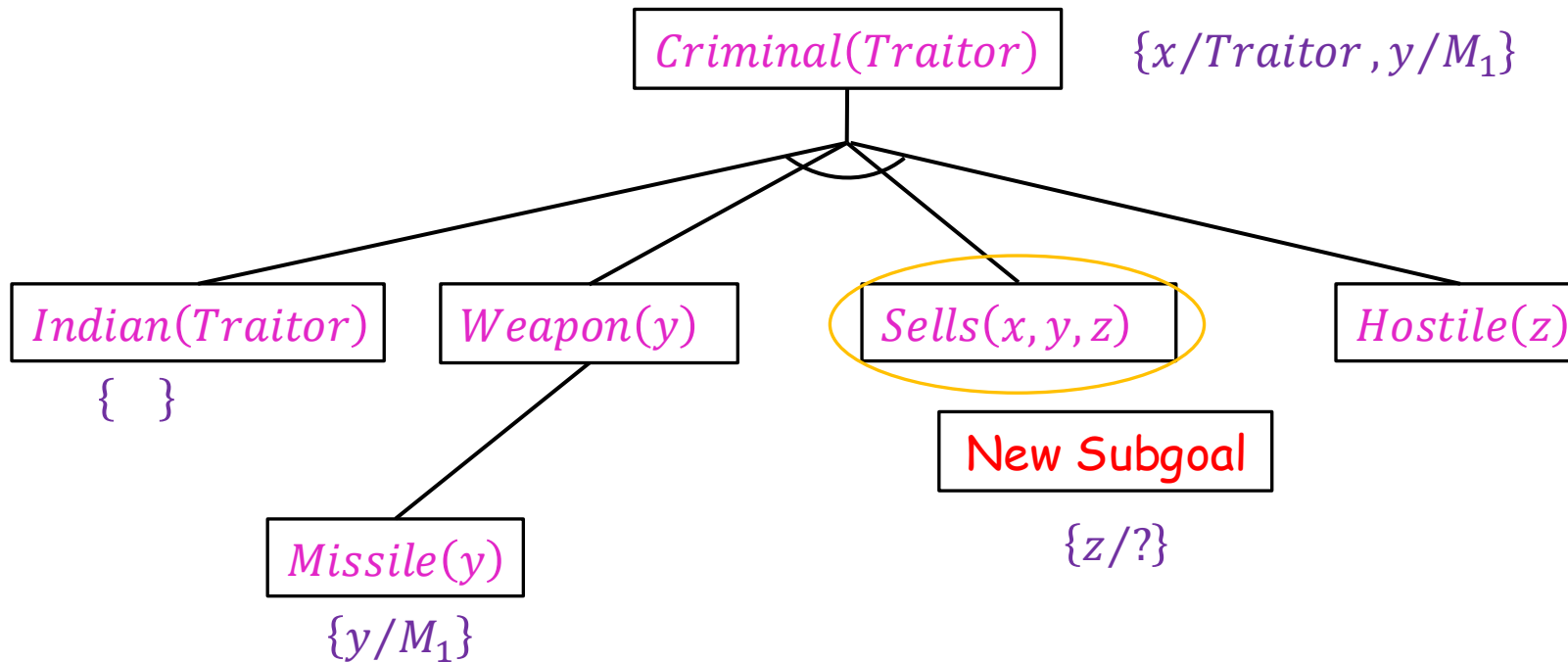
□ Depth-First Traversal

□ $KB: Missile(y) \Rightarrow Weapon(y); Missile(M_1)$



Backward Chaining Example

□ Depth-First Traversal



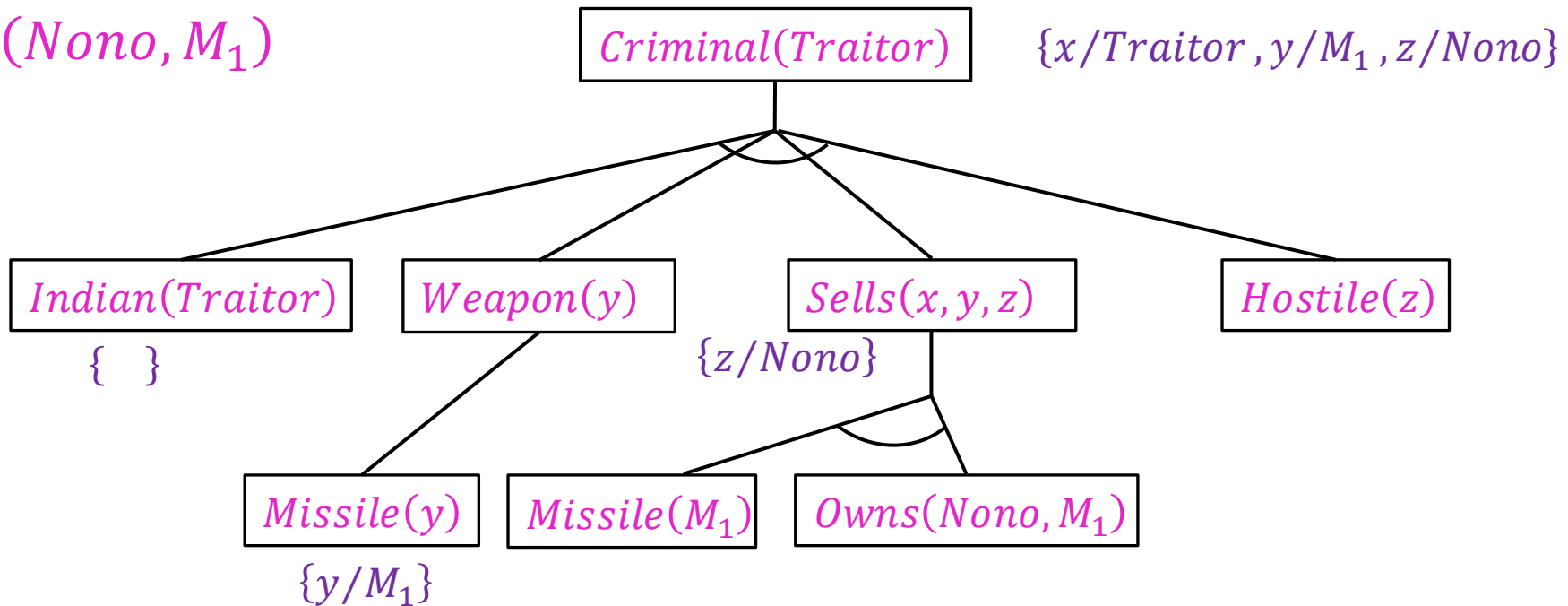
Backward Chaining Example

□ Depth-First Traversal

□ $Missile(y) \wedge Owns(Nono, y) \Rightarrow Sells(Traitor, y, Nono)$

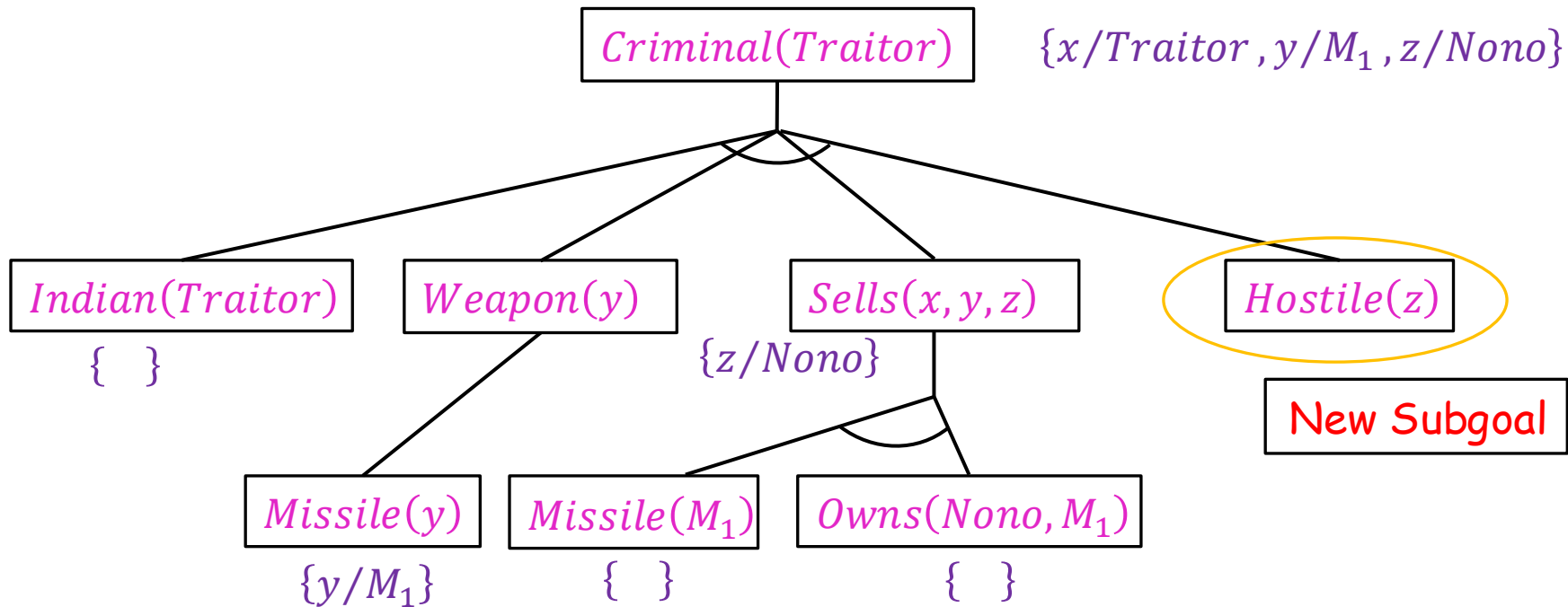
□ $Missile(M_1)$

□ $Owns(Nono, M_1)$



Backward Chaining Example

□ Depth-First Traversal

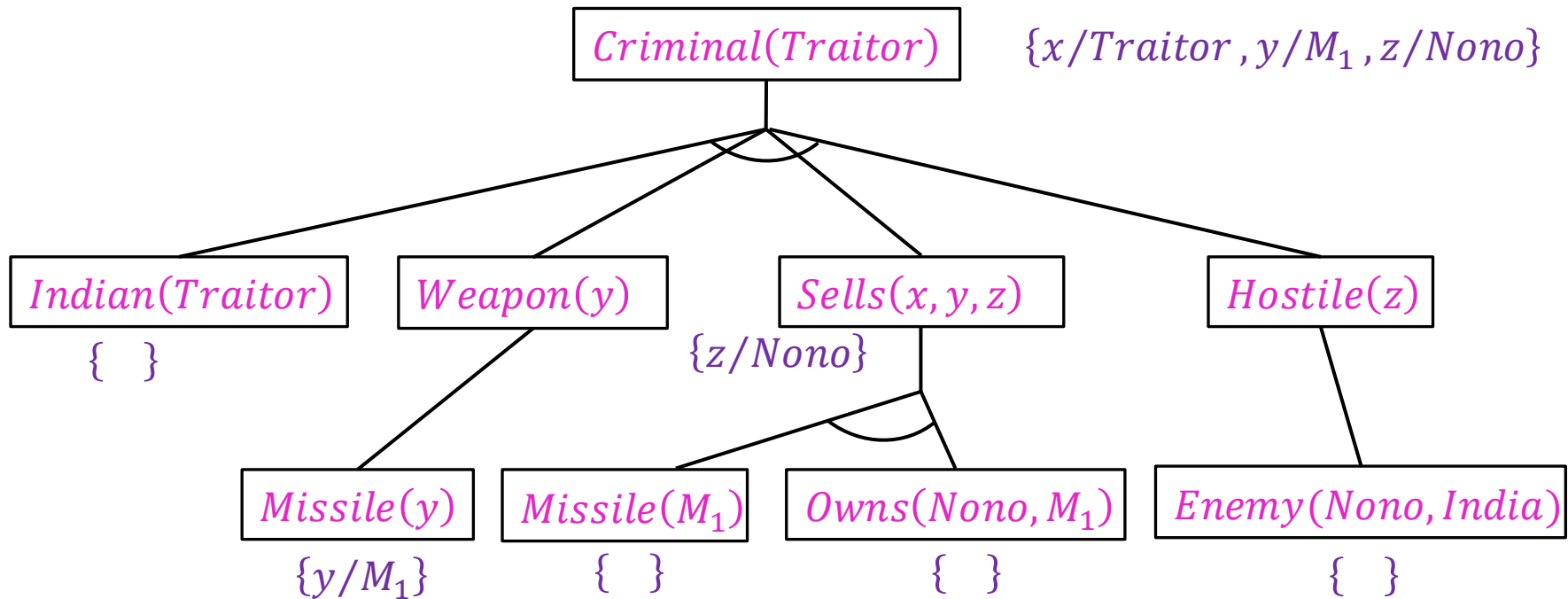


Backward Chaining Example

□ Depth-First Traversal

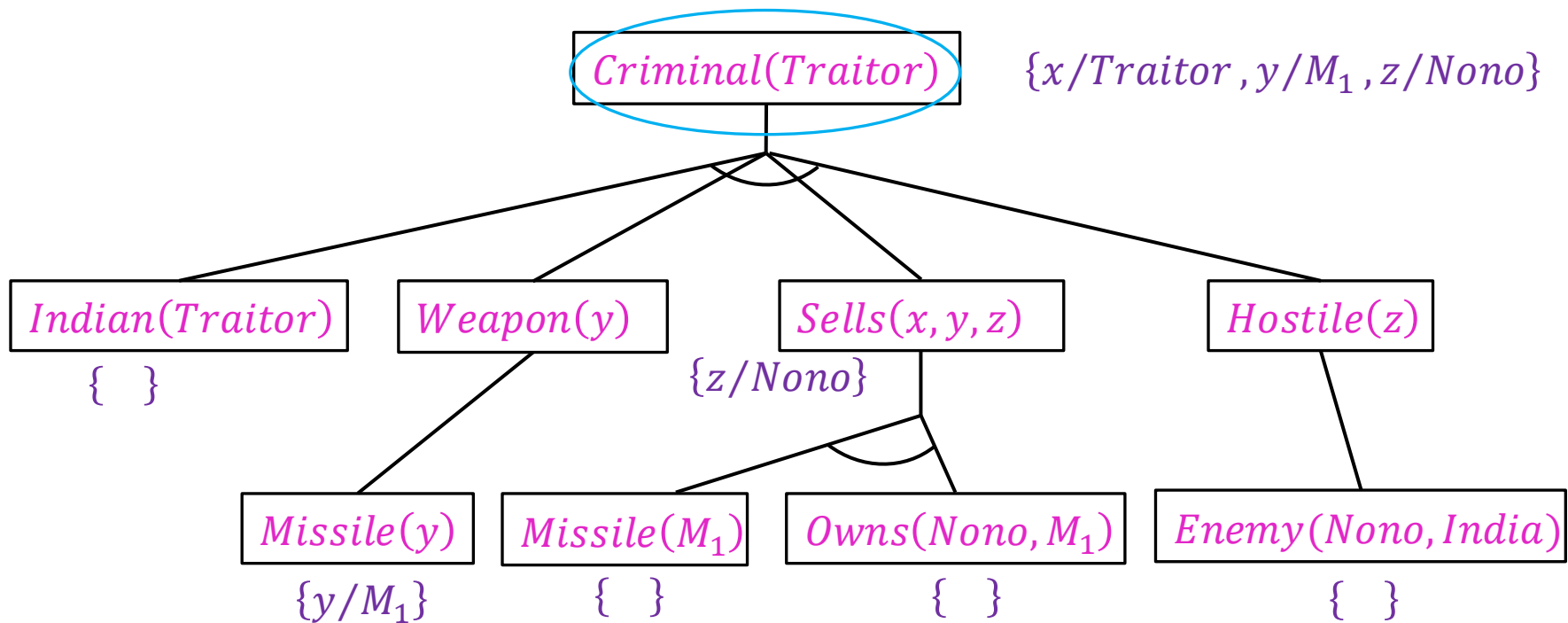
□ $KB: Enemy(z, India) \Rightarrow Hostile(z);$

□ $Enemy(Nono, India)$



Backward Chaining Example

□ Depth-First Traversal



Inference III: Backward Chaining

- ❑ Depth-First recursive search: space is linear in size of proof.
- ❑ Incomplete due to infinite loops(e.g., repeated states)
 - fix by checking current goal against all goals on stack
 - cannot fix infinite paths though
- ❑ Widely used for Logic Programming
 - PROLOG (Check Section 9.4.2-9.4.5 for more information).

Inference IV: Resolution

- ❑ Recall from propositional logic

$$\{(p \vee q), (\neg p \vee r \vee s)\} \vdash_R (q \vee r \vee s)$$

- ❑ Literal in one clause

- ❑ Its negation in the other clause

- ❑ Result is the disjunction of the remaining literals.

Inference IV: Resolution

$$\frac{\{(p(x) \vee q(A)), (\neg p(B) \vee r(x) \vee s(y))\}}{\vdash_R (q(A) \vee r(B) \vee s(y))}$$

Substitute MGU
 $\{x/B\}$ in all literals

□ In general

$$\frac{l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)\theta}$$

□ Where $Unify(l_i, \neg m_j) = \theta$

□ The two clauses are assumed to be standardized apart so that they share no variables.

□ Apply resolution steps to $CNF(KB \wedge \neg \alpha)$.

FOL: Conversion to CNF

□ Everyone who loves all animals is loved by someone

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$$

□ Steps to convert to CNF

- Eliminate biconditionals $\Leftrightarrow$ and implications $\Rightarrow$
- Move $\neg$ inwards $\neg \forall x, p \equiv \exists x \neg p, \neg \exists x p \equiv \forall x \neg p$
- **Standardize the variables**: each quantifier should use a different variable
- **Skolemize**: A more general form of existential instantiation
 - Each existential variable is replaced by a **Skolem function** of the enclosing universally quantified variables
- Drop universal quantifiers
- Distribute $\wedge$ over $\vee$

FOL: Conversion to CNF

□ It is a crime for an Indian to sell weapons to hostile nations. The country Nono, an enemy of India, has some missiles. All of its missiles were sold to it by Traitor, who is an Indian.

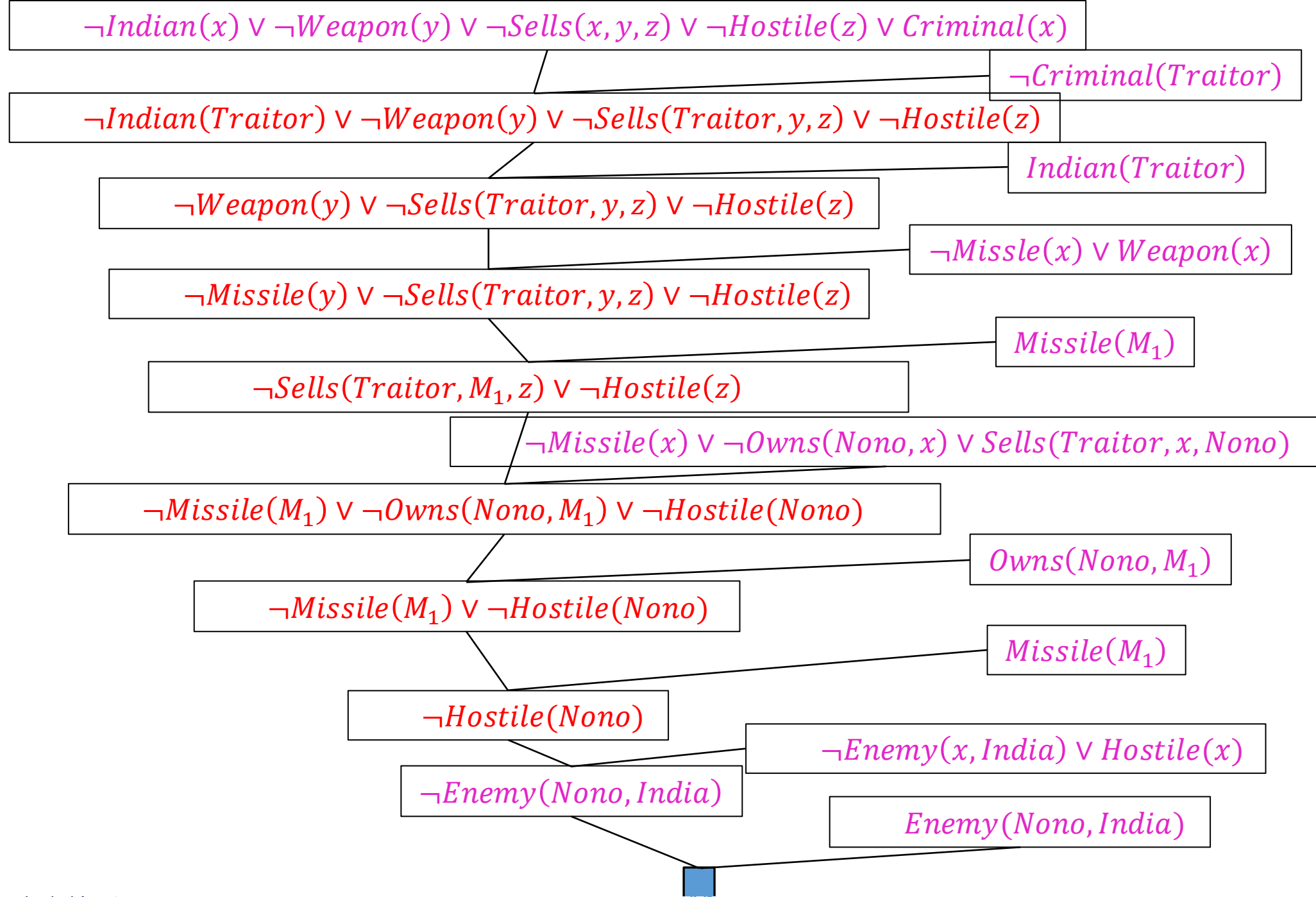
- $\neg \text{Indian}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$
- $\text{Enemy}(\text{Nono}, \text{India})$
- $\text{Owns}(\text{Nono}, M_1)$
- $\text{Missile}(M_1)$
- $\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{Traitor}, x, \text{Nono})$
- $\text{Indian}(\text{Traitor})$
- $\neg \text{Missile}(x) \vee \text{Weapon}(x)$
- $\neg \text{Enemy}(x, \text{India}) \vee \text{Hostile}(x)$

Variables are not standardized here.

□ Resolution uses “proof by contradiction”

- Show $KB \models \alpha$ by showing $KB \wedge \neg \alpha$ is unsatisfiable.

□ To prove, Traitor is a criminal. Add $\neg \text{Criminal}(\text{Traitor})$ to KB and derive empty clause.



FOL Resolution Example (2)

❑ Everyone who loves all animals is loved by someone

$$\forall x[\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$$

❑ Anyone who kills an animal is loved by no one

$$\forall x[\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z)] \Rightarrow [\forall y \neg \text{Loves}(y, x)]$$

❑ Jack loves all animals

$$\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$$

❑ Either Jack or Curiosity killed the cat, who is named Tuna

$$\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$$

❑ Did Curiosity kill the cat?

$$\text{Kills}(\text{Curiosity}, \text{Tuna})?$$

$$\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$$

$$\text{Cat}(\text{Tuna})$$

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$$

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]$$

$$\forall x [\exists y \neg (\neg \text{Animal}(y) \vee \text{Loves}(x, y))] \vee [\exists y \text{ Loves}(y, x)]$$

$$\forall x [\exists y (\text{Animal}(y) \wedge \neg \text{Loves}(x, y))] \vee [\exists y \text{ Loves}(y, x)]$$

$$\forall x [\exists y (\text{Animal}(y) \wedge \neg \text{Loves}(x, y))] \vee [\exists z \text{ Loves}(z, x)]$$

$$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$$

$$[\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$$

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)]$$

$$\forall x [\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z)] \Rightarrow [\forall y \neg \text{Loves}(y, x)]$$

$$\forall x [\neg \exists z \text{ Animal}(z) \wedge \text{Kills}(x, z)] \vee [\forall y \neg \text{Loves}(y, x)]$$

$$\forall x [\forall z \neg (\text{Animal}(z) \wedge \text{Kills}(x, z))] \vee [\forall y \neg \text{Loves}(y, x)]$$

$$\forall x [\forall z \neg \text{Animal}(z) \vee \neg \text{Kills}(x, z)] \vee [\forall y \neg \text{Loves}(y, x)]$$

$$\neg \text{Animal}(z) \vee \neg \text{Kills}(x, z) \vee \neg \text{Loves}(y, x)$$

$$\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$$

$$\forall x \neg \text{Animal}(x) \vee \text{Loves}(\text{Jack}, x)$$

$$\neg \text{Animal}(x) \vee \text{Loves}(\text{Jack}, x)$$

$$\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$$

$$\forall x \neg \text{Cat}(x) \vee \text{Animal}(x)$$

$$\neg \text{Cat}(x) \vee \text{Animal}(x)$$

